Die Wette: 1000 Zahlen, zehn Fragen

Binäre Suche — halbieren statt durchgehen

AUSGANGSSITUATION



Lea und Tim wetten um einen Döner. Lea: „Denk dir eine Zahl zwischen 1 und 1000. Ich finde sie mit **höchstens zehn Fragen** – du antwortest nur mit ‚größer', ‚kleiner' oder ‚richtig'." Tim: „Niemals. Das sind tausend Zahlen!" Lea hat noch eine zweite Idee: Mit demselben Trick will sie im Sekretariat prüfen, ob eine Schülerausweis-Nummer gültig ist. Die Liste der gültigen Nummern hat **1000 Einträge** und ist aufsteigend sortiert.

Wer gewinnt die Wette – und wie viele Vergleiche braucht Leas Trick in der Ausweisliste im schlimmsten Fall?

Was tippst du? Kreuze an, bevor du rechnest:

☐ Tim gewinnt ☐ Lea gewinnt ☐ kommt auf die Zahl an

Ausschnitt aus der sortierten Ausweisliste – 15 Nummern (gesetzte Annahme)

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Nummer	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678

1 Aufgabe 1 — Aufwärmen

Nenne aus dem Gedächtnis die Anzahl der Vergleiche, die die lineare Suche aus der letzten Stunde in einer Liste mit 1000 Einträgen im schlimmsten Fall braucht.

SO GEHT'S

So sucht ein Rechner in einer sortierten Liste: die binäre Suche

1. Der Suchbereich ist zuerst die ganze Liste: **links** = 0, **rechts** = letzter Index.
2. Berechne die Mitte: mitte = (links + rechts) // 2 – dabei wird abgerundet.
3. Vergleiche den Wert an der Mitte mit dem gesuchten Wert: gleich → gefunden. Kleiner als gesucht → suche rechts weiter: links = mitte + 1. Größer als gesucht → suche links weiter: rechts = mitte − 1.
4. Wiederhole ab Schritt 2, solange links ≤ rechts gilt.
5. Wird links größer als rechts, ist der Bereich leer: Der Wert **fehlt**.

BEISPIEL

Beispiel: Suche nach 1349

Suche nach 1349. links 0, rechts 14, mitte 7 → 1701 ist größer als 1349, also rechts = 6. Neue Mitte (0 + 6) // 2 = 3 → 1349 gefunden. **2 Vergleiche** statt 4 bei der linearen Suche.

2 Aufgabe 2

Vervollständige das Protokoll für die Suche nach der Nummer **2417**. Der erste Schritt ist eingetragen. Trage darunter die Anzahl der Vergleiche und den Index ein.

Schritt	links	rechts	mitte	Nummer an mitte	kleiner / größer / gleich?	neuer Bereich
1	0	14	7	1701	kleiner als 2417	links = 8
2						
3						
4						
5						

Anzahl der Vergleiche: _____ 2417 steht bei Index: _____

3 Aufgabe 3

a) **Kennzeichne** für die Suche nach der Nummer **1900** in jedem Schritt den Bereich, der noch durchsucht wird (Rahmen um die Kästchen), und kreise die Mitte ein. Nicht gebrauchte Zeilen bleiben leer.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Schritt 1	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 2	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 3	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 4	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678
Schritt 5	1012	1158	1207	1349	1420	1585	1633	1701	1876	1988	2105	2233	2417	2560	2678

b) **Erkläre**, woran der Algorithmus erkennt, dass 1900 nicht in der Liste steht.

MERKE

Halbieren zählen

Jeder Vergleich **halbiert** den Suchbereich. Wie viele Vergleiche die binäre Suche höchstens braucht, verrät die Frage: Wie oft kann man n halbieren, bis nur noch ein Eintrag übrig ist? Suche dafür das kleinste k, für das $\mathbf{2^k \geq n}$ gilt.

$$2^4 = 16 \cdot 2^7 = 128 \cdot 2^{10} = 1024 \cdot 2^{20} \approx 1 \text{ Million}$$

Beispiel: n = 15 Nummern: $2^4 = 16 \geq 15$, also höchstens 4 Vergleiche. n = 100: $2^7 = 128 \geq 100$, also höchstens 7.

4 Aufgabe 4

Berechne, wie viele Vergleiche die binäre Suche im schlimmsten Fall braucht – für Leas Zahlenwette (1000 Zahlen) und für eine Kundendatei mit 1 Million Einträgen. Trage zum Vergleich daneben ein, was die lineare Suche braucht.

n	kleinstes k mit $2^k \geq n$	binäre Suche: Vergleiche	lineare Suche: Vergleiche
1000			
1 000 000			

5  Aufgabe 5 — an den Rechner

Algorithmen-Werkstatt → Binäre Suche → **Selbst machen**. Du klickst die Mitte an und entscheidest, ob es links oder rechts weitergeht – die Website prüft jeden Schritt.

`https://philipp-byte.github.io/algorithmen-werkstatt/binaere-suche.html`

Dokumentiere zwei Suchläufe mit einer Zufallsliste der Website: den gesuchten Wert, den gefundenen Index (oder „fehlt“) und die Anzahl der Vergleiche.

Lauf 1 – gesucht: _____ Index: _____ Vergleiche: _____

Lauf 2 – gesucht: _____ Index: _____ Vergleiche: _____

INFO

Vom Ablauf zum Python-Code

Eine `while`-Schleife läuft, **solange** ihre Bedingung wahr ist – hier also, solange der Suchbereich nicht leer ist. Die Variablen `links` und `rechts` werden in der Schleife verändert; irgendwann wird die Bedingung falsch.

`//` ist die **Ganzzahl-Division**: `7 // 2` ergibt 3, nicht 3,5. So bleibt die Mitte ein gültiger Index.

Mit `elif` prüft Python eine zweite Bedingung nur dann, wenn die erste falsch war.

6 Aufgabe 6

Implementiere die fehlenden Teile der Funktion. Sie soll den Index der gesuchten Nummer zurückgeben – oder −1, wenn die Nummer fehlt.

```
1  def binaere_suche(liste, gesucht):
2      links = 0
3      rechts = len(liste) - 1
4      while _________________ :
5          mitte = (links + rechts) ______ 2
6          if liste[mitte] == gesucht:
7              return mitte
8          elif liste[mitte] < gesucht:
9              links = _________
10         else:
11             rechts = _________
12     return _______
13
14 ausweise = [1012, 1158, 1207, 1349, 1420, 1585, 1633, 1701, 1876, 1988, 2105, 2233, 2417, 2560, 2678]
15 print(binaere_suche(ausweise, 2417))
```

Wenn du nicht weiterkommst: Welche Bedingung steht in Schritt 4 des Infokastens – und was passiert in Zeile 9 mit dem Wert an der Mitte, der gerade verglichen wurde?

7 Aufgabe 7 — am Rechner

Teste deine Funktion in der Werkstatt unter **Code schreiben** (oder in Thonny) mit der Ausweisliste. Trage die Ausgaben ein und kreuze an, wenn alle Tests bestanden sind.

Ausgabe für 2417: _____ Ausgabe für 1900: _____ ☐ alle Tests bestanden



Bevor du schreibst: Die Wette geht über **1000 Zahlen**, nicht über die 15 aus dem Ausschnitt. Welche Zeile aus Aufgabe 4 brauchst du dafür?

8 Aufgabe 8 — Die Wette

Begründe mit deinen Ergebnissen aus Aufgabe 4, wer die Wette gewinnt – und warum Leas Trick bei Mias unsortierter Ticketliste aus der letzten Stunde nicht funktioniert. Vergleiche am Ende mit deinem Tipp vom Anfang.

Satzanfänge: „… gewinnt, weil bei 1000 Zahlen höchstens …“ · „Nach jeder Frage bleibt nur noch …“ · „Bei einer unsortierten Liste weiß Lea nach der Antwort ‚größer‘ nicht, …“

9 Aufgabe 9 — Vertiefung

Beurteile, ob Lea die Wette noch gewinnen kann, wenn Tim die Regeln ändert: Er antwortet nur noch mit „ja“ oder „nein“ auf Fragen wie „Ist es die 512?“.

10 Aufgabe 10 — Zusatz

Entwickle ein Vorgehen für das Sekretariat: Die 1000 Ausweisnummern liegen dort in der Reihenfolge der Ausgabe vor – unsortiert. Die Sekretärin prüft am Tag etwa 500 Nummern. Halte fest, welche Schritte das Programm einmalig und welche es bei jeder Prüfung ausführen soll, und gib grob an, wie viele Vergleiche pro Tag dein Vorgehen spart.

PRÜFE DICH SELBST

- [ ] Ist die Mitte in deinem Protokoll aus Aufgabe 2 immer abgerundet – nie 11,5?
- [ ] Wird dein Bereich in Aufgabe 3 in jedem Schritt kleiner, bis er leer ist?
- [ ] Liefert deine Funktion für 1900 die Zahl −1 – und läuft nicht endlos?